

S1.

비트를 다루는 네 가지 도구

AND · OR · NOT · XOR

이 섹션의 도착

네 연산의 진리표를 채우고,
8비트 예측 4문제 중 3문제 이상 적중한다

워크시트 · 실습 1 · 논리 연산 예측 적중표

도 입

그 숫자는 어떻게 움직이는가

어제까지: 모든 것은 숫자였다 (Day 2) · 오늘: 그 숫자를 거르고, 켜고, 뒤집는 도구 4개

개 념

네 가지 질문

AND — 둘 다 1인가?

하나라도 0이면 꺼진다.
용도: 거르기(마스크)

OR — 하나라도 1인가?

둘 다 0일 때만 꺼진다.
용도: 켜기(세트)

XOR — 서로 다른가?

같으면 0.
용도: 토글·비교

NOT — 뒤집기

입력이 하나뿐.
용도: 반전

개 념

진리표 — 오늘의 구구단

A	B	AND	OR	XOR	NOT A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

이 표는 지금 보여주되, 실습에서는 가리고 예측하게 함을 예고.

시연

8비트에 한꺼번에

- 자리마다 독립 — 덧셈과 달리 자리올림이 없다
- $204 \& 15 = 12$ — B가 1인 자리만 통과 (하위 4비트 추출)
- 시연은 이 한 문제만 — 나머지 3문제는 여러분 몫

▶ 논리 연산 실험대

anim/logic_gates_bitwise.html

논리 연산 예측 적중표

- ① 진리표 16칸 예측 (표는 가리고)
- ② 8비트 4문제 예측 — 2진 8자리 + 10진
- ③ 채점하기 → 예측 칸 잠김
- ④ 애니메이션 프리셋으로 눈 검사
- ⑤ 한 줄 정리 작성

결과물

예측 적중표
(24칸 채점 완료 화면)

▶ 논리 연산 실험대

▶ 워크시트 열기

예측 먼저, 화면은 검사 — 채점하면 예측 칸이 잠깁니다

확 인

손 전체 응답 — 번호를 손가락으로

“ 8비트 4문제 — ① 다 맞았다 ② 1개 이상 틀려서 고쳤다 ③ 모르겠다 — 번호로 ”

② 가 많으면 → ② 다수 문항을 애니메이션으로 다 같이 재실행

가장 많이 나올 오답

오 답

XOR에서 $(1, 1) \rightarrow 1$

왜: OR('하나라도?')과 혼동



교 정 화 법 — 토 씨 그 대 로

" XOR의 질문은 '다른가?'입니다. 1과 1은 같으니까 0. OR의 질문과 소리 내어 구분해 보세요. "

다음 시간

이 게이트 4개가 부품의 전부 —
다음 시간, 이걸로 덧셈하는 기계를 조립합니다

S2.

게이트로 덧셈기 조립

반가산기 · 전가산기

이 섹션의 도착

반가산기 진리표를 예측하고,
검산식 $2 \times \text{Cout} + \text{Sum}$ 이 입력 합과 일치함을 확인한다

워크시트 · 실습 2 · 가산기 조립 확인표

곤란한 순간은 단 하나 — $1 + 1 = 10_2$

십진수의 $7+8=15$ 와 같은 일: '5를 쓰고, 1을 올린다'. 답을 이 자리(Sum)와 윗자리(Carry)로 쪼갬다

시연

반가산기 — 게이트 딱 2개

- $\text{Sum} = \text{XOR}$: '둘이 다른가?' → 이 자리에 쓸 것
- $\text{Carry} = \text{AND}$: '둘 다 1인가?' → 윗자리로 올릴 것
- 곤란한 행은 $1+1$ 하나뿐 — 그 행을 위해 존재한다

▶ 반가산기

[anim/half_adder.html](#)

개 념

왜 '반(半)'인가

bit 0에서는 완벽

아랫자리가 없으니 입력 2개면 충분

bit 1부터는 부족

아랫자리 올림 Cin이 들어올 문이 없다
(회로도의 빨간 점선)

해결 = 이어 붙이기

반가산기 2개 + OR 1개 → 전가산기

시연

전가산기 — 1을 세는 기계

- $\text{Sum} = 1$ 이 홀수 개인가 (XOR 두 번)
- $\text{Cout} = 1$ 이 2개 이상인가 (다수결)
- 검산: $A + B + \text{Cin} = 2 \times \text{Cout} + \text{Sum}$
- 8개 이르면 8비트 덧셈기 — 맨 왼쪽 Cout이 곧 CF

▶ 전가산기

[anim/full_adder.html](#)

가산기 조립 확인표

- ① 반가산기 진리표 예측 → 채점(잠김)
- ② 전가산기 (1,0,1)·(1,1,1) 실행하며 기입
- ③ 검산식 $2 \times \text{Cout} + \text{Sum}$ 확인
- ④ 빈칸 채우기 + '반인 이유' 서술

결과물

가산기 확인표
(20칸 채점 완료 화면)

▶ [반·전가산기](#)

▶ [워크시트 열기](#)

예측 먼저, 화면은 검산 — 채점하면 예측 칸이 잠깁니다

확 인

손 전체 응답 — 번호를 손가락으로

“ 검산식 — ① 두 줄 다 입력 합과 일치 ② 하나만 일치 ③ 모르겠다 — 번호로 ”

② 가 많으면 → (1,1,1)을 전체 화면으로 재실행하며 $2 \times 1 + 1 = 3$ 확인

가장 많이 나올 오답

오 답

Carry를 만드는 게이트 = OR

왜: '올림 후보 합치기'의 OR과 혼동



교 정 화 법 — 토 씨 그 대 로

" OR은 마지막에 후보 둘을 합칠 때만 씁니다. '둘다 1인가'는 AND의 질문입니다. "

다음 시간

덧셈기는 완성.

그런데 뺄셈기는 어디에? — 없습니다. 다음 시간에 그 이유를.

S3.

뺄셈 회로는 없다

2의 보수

이 섹션의 도착

9-13을 반전 → +1 → 덧셈 → 버림의 전 과정으로 풀고
애니메이션과 전 행 일치시킨다

워크시트 · 실습 3 · 2의 보수 뺄셈 검산지

선 언

CPU 안에 뿔셈 회로는 없다

방금 조립한 덧셈기 하나로 뿔셈까지 합니다 — 빼는 수를 '더할 수 있는 꼴'로 바꿔서

개 념

-B 만드는 법 — 세 동작

① 전부 뒤집는다

$\sim B$ — S2에서 배운 NOT
(1의 보수)

② 1을 더한다

$\sim B + 1 = 2$ 의 보수 = $-B$ 역할

③ 그냥 더한다

$A + (-B)$
넘친 9번째 비트는 버림 (mod 256)

왜 성립하는가

$$B + \sim B = 255 \rightarrow +1 \text{이면 } 256 = 0$$

8비트 세계에서 256은 0. 그래서 $\sim B + 1$ 은 정확히 $-B$ 역할 — 버려진 256이 빌려온 256과 상쇄되어 장부가 맞는다

시연

시연 — 7 - 3

- STAGE 1: B=3을 뒤집고 +1 → 253 (-3 역할)
- STAGE 2: $7 + 253 = 260 \rightarrow$ 넘친 1을 버림 → 4
- 여러분의 문제 9-13은 버림이 없는 경우 — 왜인지는 실습에서

▶ 2의 보수 뺄셈

anim/twos_complement_subtraction.html

2의 보수 뺄셈 검산지

- ① 애니메이션 닫기 — 손 먼저
- ② 9-13 전 행 손 계산 (반전→+1→뺄셈→버림→10진)
- ③ 채점(잠김) → 애니메이션에 9.13 넣어 행별 대조
- ④ 자유 문제(결과가 음수) 출제·검증

결과물

검산지

(지정+자유 문제 완료 화면)

▶ 2의 보수 뺄셈

▶ 워크시트 열기

예측 먼저, 화면은 검산 — 채점하면 예측 칸이 잠깁니다

확 인

손 전체 응답 — 번호를 손가락으로

“ 9-13 — ① 전 행 일치 ② 일부 틀려서 고쳤다 ③ 모르겠다 — 번호로 ”

② 가 많으면 → 9-13을 전체 화면으로 행별 재생

가장 많이 나올 오답

오 답

**+1을 빼먹은 1의 보수 계산
(9-13이 -5로 나옴)**

왜: '뒤집고'에서 멈춤



교 정 화 법 — 토 씨 그 대 로

“ 거기서 멈추면 정확히 1이 모자랍니다. -B가 되려면 반드시 +1 — 그래서 이름이 2의 보수. ”

다음 시간

점심 후 — 계산기는 답만 주지 않습니다.
쪽지도 남깁니다.

S4.

연산이 남기는 신호

상태 플래그 CF · ZF · SF · OF

이 섹션의 도착

여섯 사례의 플래그 24칸을 예측해 18칸 이상 적중하고,
CF와 OF의 차이를 한 문장으로 쓴다

워크시트 · 실습 4 · 상태 플래그 예측표

버려진 그 1은 어디로 갔을까

버려지지 않았습니다 — CF라는 쪽지로 남습니다. 계산기는 답과 함께 신호등 4개를 켵니다

개 념

신호등 4개

CF — 자리올림/빌림

부호 없는 세계의 경고
(255를 넘거나, 빌려왔거나)

ZF — 결과가 0인가

8비트가 전부 0일 때

SF — 최상위 비트

부호 있는 해석으로 음수인가

OF — 범위 초과

부호 있는 세계의 경고
(-128 ~ 127을 벗어남)

같은 비트, 두 가지 읽기

$$200 + 100 \rightarrow CF = 1, OF = 0$$

부호 없는 눈: 300은 8비트에 못 담음 $\rightarrow$ 경고 . 부호 있는 눈: $(-56) + 100 = 44$, 범위 안 $\rightarrow$ 문제 없음

시연

시연 순서

- $255 + 1 \rightarrow \text{CF} \cdot \text{ZF}$ (한 바퀴 돌아 0)
- $200 + 100 \rightarrow \text{CF}$ 만 — 판정 패널 정독
- $128 + 128$ 은 실습 후 다 같이 (셋이 동시에 커짐)

▶ 플래그 시뮬레이터

anim/cpu_flags_simulator.html

실습 4

상태 플래그 예측표

- ① 여섯 사례 24칸을 전부 예측
- ② 채점하기 → 예측 잠김
- ③ 프리셋 6개를 눌러 틀린 사례 재확인
- ④ CF/OF 구분 + '200+100' 이유 서술

결과물

플래그 예측표
(26칸 채점 완료 화면)

▶ [플래그 시뮬레이터](#)

▶ [워크시트 열기](#)

예측 먼저, 화면은 검산 — 채점하면 예측 칸이 잠깁니다

확 인

손 전체 응답 — 번호를 손가락으로

“ 24칸 중 — ① 18칸 이상 맞았다 ② 그보다 적어서 다시 봤다 ③ 모르겠다 — 번호로 ”

② 가 많으면 → 128+128을 전체 화면으로 재실행 (CF·ZF·OF 동시 점등)

가장 많이 나올 오답

오 답

200 + 100에서 OF = 1

왜: CF와 OF를 같은 경고로 취급



교 정 화 법 — 토 씨 그 대 로

“ 부호 있는 눈으로 다시 읽어 보세요 — 200은 -56입니다. $-56+100=44$, 범위 안. OF는 부호 있는 세계의 눈으로만 판단합니다. ”

다음 시간

지금까지 플래그를 읽었다면 —
다음 시간엔 비트를 직접 씁니다. 진짜 장치를 겁니다.

S5.

비트로 장치를 켜다

하드웨어 플래그 제어

이 섹션의 도착

켜기 / 끄기 / 토글 / 검사를 C 코드 한 줄씩으로 쓰고,
애니메이션 코드 로그와 글자까지 일치시킨다

워크시트 · 실습 5 · 하드웨어 제어 코드 옮겨쓰기

도입

상태 vs 제어 — 방향이 반대

상태 플래그 (S4)

CPU가 쓰고, 우리가 읽는 쪽지

제어 레지스터 (지금)

우리가 쓰면, 장치가 움직인다
비트 하나 = 장치 하나

마스크 만들기

1 << n

숫자 1을 왼쪽으로 n칸 — 원하는 자리에만 1이 있는 도장. 애니메이션에서 1이 한 칸씩 점프하는 그 장면

개 념

네 가지 관용구

할 일	코드	쓰는 연산
켜기	$\text{REG} = (1 \ll n);$	OR
끄기	$\text{REG} \&= \sim(1 \ll n);$	AND + NOT
토글	$\text{REG} \wedge= (1 \ll n);$	XOR
검사	$\text{REG} \& (1 \ll n)$	AND (읽기만)

원칙 한 줄: 읽고 – 비트만 고치고 – 다시 쓴다. 다른 7비트는 보존.

시연

시연 — 부팅 시퀀스

- 시나리오 자동 실행으로 전체 그림 (장치가 차례로 켜짐)
- 히터 켜기 1건을 3단계로 정독: 마스크 → 연산 → 반영
- 실습 전 반드시 새로그침 — REG = 00000000 에서 시작

▶ 하드웨어 플래그 제어

anim/hardware_flag_control.html

실습 5

하드웨어 제어 코드 옮겨쓰기

- ① 새로그침 → '과열 대응' 실행·관찰
- ② 코드 로그를 보지 않고 3줄 작성
- ③ 채점 → 로그와 글자까지 대조
- ④ 내 시나리오 3줄 → 수동 조작으로 검증

결과물

제어 코드 3줄
(로그 대조 완료 화면)

▶ [하드웨어 제어](#)

▶ [워크시트 열기](#)

예측 먼저, 화면은 검산 — 채점하면 예측 칸이 잠깁니다

확 인

손 전체 응답 — 번호를 손가락으로

“ 코드 3줄 — ① 로그와 전부 일치 ② 일부 달라서 고쳤다 ③ 모르겠다 — 번호로 ”

② 가 많으면 → 꼬기 한 줄을 다 같이 타이핑 리듬으로 (앤드-이퀄-틸드)

가장 많이 나올 오답

오 답

끄기 = REG &= (1 << 3);

왜: ~(NOT) 누락



교 정 화 법 — 토 씨 그 대 로

“ 그 마스크로 AND하면 히터만 남고 나머지가 전부 꺼집니다 — 정반대죠. 끄려면 뒤집은 마스크 $\sim(1 << 3)$. ”

다음 시간

이 조작이 뒤에 배울 MCU 과목의 일상입니다.
그리고 다음 시간 — 드디어 CPU가 명령을 읽습니다.

S6.

컴퓨터가 알아듣는 최소 단위

명령과 인출 – 해석 – 실행

이 섹션의 도착

**LOAD→ADD→STORE→HALT 네 명령을
인출-해석-실행 박자로 손 추적해 추적표 17칸을 채운다**

워크시트 · 실습 6 · 명령 실행 추적표

메모리 칸의 26 — 데이터인가, 명령인가

명령어도 그냥 숫자입니다. 정하는 것은 '그 숫자가 언제, 어디로 읽히느냐'뿐

시연

주소 접근 — 인출의 밑바닥 (시연만)

- 어느 칸이든 같은 박자 — 그래서 Random Access
- '자동 방문'으로 확인: 구석이든 가운데든 똑같다
- 산출물 없음 — 눈으로만

[강사 확정] 주소 애니메이션 택1

· cpu_memory_addressing (절반 거르기)

· memory_row_column_addressing (날줄·씨줄)

비전공 비중 높으면 후자 권장

개 념

세 박자

① 인출

PC가 가리키는 칸을 IR로.
아직 뜻은 모른다 · PC는 +1

② 해석

상위 4비트=무엇을, 하위 4비트=어디
에.
숫자가 명령이 되는 순간

③ 실행

LOAD / ADD / STORE / HALT.
ADD는 플래그를 남긴다 (S4)

시연

시연 — 명령 1개만

- LOAD 한 개를 세 박자로 정독 (값이 날아가는 장면)
- 나머지 세 명령은 종이 카드 손 실습 몫
- ADD의 플래그가 '없음'인 이유는 실습에서 발견

▶ 명령 실행 사이클

anim/cpu_instruction_cycle.html

실습 6

명령 실행 추적표

- ① 종이 카드로 4개 명령 순 추적
- ② 추적표 17칸 작성 (PC·IR·뜻·A·플래그)
- ③ 채점
- ④ '다음 단계 ▶'로 박자별 대조

결과물

명령 추적표
(17칸 채점 완료 화면)

▶ 명령 실행 사이클

▶ 워크시트 열기

예측 먼저, 화면은 검산 — 채점하면 예측 칸이 잠깁니다

확 인

손 전체 응답 — 번호를 손가락으로

“ 추적표 — ① 전부 일치 ② 일부 고쳤다 ③ 모르겠다 — 번호로 ”

② 가 많으면 → 명령 2(ADD)를 전체 화면으로 세 박자 재생

가장 많이 나올 오답

오 답

인출 후 PC = 0, 1, 2, 3

왜: PC가 '지금'을 가리킨다고 오해



교 정 화 법 — 토 씨 그 대 로

" PC는 '다음'을 가리키는 손가락입니다. 가져오는
그 순간 이미 +1. "

다음 시간

다음 시간은 시험이 아니라 도착점 —
이번엔 숫자를 여러분이 고릅니다.

S7.

도착점 실습

내가 고른 수로 프로그램 실행

이 섹션의 도착

**CPU 시뮬레이터에서 덧셈 프로그램을 명령어 단위로 실행하며
레지스터·메모리 값의 변화를 단계별로 짚어 보인다**

워크시트 · 실습 7 · 시뮬레이터 단계별 기록

규 칙

실습 규칙

두 수는 자유

0~255에서 직접 선택.
채점도 당신이 고른 수 기준으로 자동

2회차 조건

합이 255를 넘는 쌍 —
CF를 내 손으로 킂다

조건 미달이면

워크시트가 되돌려 보냅니다.
다른 쌍으로 다시

1 회 차

고른 수 → 단계 실행 → 기록 → 채점

습관 하나: 다음 스텝에서 A가 몇이 될지 먼저 말하고 누르기

합 > 255 → CF 관찰

켜진 이유를 한 줄로: '9번째 비트로 넘쳐서'. 오전 S3의 버림, S4의 CF가 지금 내 손에서 재현된다

시연

마무리

- HALT 화면 캡처 2장 (1·2회차)
- 빨리 끝나면: 부록 float32 또는 'ZF가 켜지는 쌍' 찾기
- 강사 순회 — 화면을 직접 확인합니다

▶ 명령 실행 사이클

anim/cpu_instruction_cycle.html

시뮬레이터 단계별 기록

- ① 1회차: 수 선택 → 실행 → 기록 → 채점
- ② 2회차: 합>255 쌍 → CF 확인
- ③ 'CF가 켜진 이유' 한 줄
- ④ HALT 캡처 2장

결과물

단계별 기록
(동적 채점 완료 화면)

▶ 명령 실행 사이클

▶ 워크시트 열기

예측 먼저, 화면은 검산 — 채점하면 예측 칸이 잠깁니다

확 인

손 전체 응답 — 번호를 손가락으로

“ 2회차 CF — ① 켜졌다 ② 안 켜져서 쌍을 바꿨다 ③ 모르겠다 — 번호로 ”

② 가 많으면 → ③가 있으면 즉시 1:1 — 이 확인이 도착점 직전 관문

가장 많이 나올 오답

오답

화면을 보며 베끼기만 (예측 없이 기록)

왜: 실습을 받아쓰기로 오해



교정 화법 — 토씨 그대로

“ 다음 스텝에서 A가 몇이 될지 먼저 말해 보고 누르세요 — 예측이 오늘 기른 근육입니다. ”

다음 시간

마지막 시간 — 이 기록을 들고 옆 사람에게 설명합니다.
그것이 3일의 도착점입니다.

S8.

종합 — 짝 시연과 3일 회수

도착점 완료와 제출

이 섹션의 도착

자기 기록표를 보며 실행 과정을 짝에게 1분 설명하고,
도착점 완료 화면을 제출한다

짝 시연

짝 시연 규칙

기록표만 본다

화면은 짝이 조작 — 설명자는 손 대지 않기

1분 + 피드백 30초

'어느 박자 설명이 가장 명확했나'
한 가지만

종료 시

서로의 워크시트 ③에
짝 이름 기입 · 체크

제출

도착점 완료 화면 → 캡처 → 노선

실습 1~6 점수가 자동 수집된 요약이 함께 찍힙니다 — 별도 취합 없음. 강사는 노선에서 인원수만 확인

회수

3일 회수 — 한 장의 그림

Day 1 — 부품 (그릇)

CPU · 메모리 · 저장장치 · 입출력

Day 2 — 숫자 (내용물)

2진수 · 문자 · 색 · 16진수

Day 3 — 명령 (움직임)

게이트 → 가산기 → 뿔셈 → 플래그
→ 제어 → 명령 → 실행

예 고

다음 과목으로 가는 다리

운영체제 활용

오늘 실행한 프로그램 수백 개를
누가 교통정리하나

MCU 동작 원리

오늘의 S5(레지스터 제어)가
일상이 되는 곳

짜к 시연과 제출

- ① 교대로 1분 설명 + 30초 피드백
- ② 짜к 이름 기입 · 체크
- ③ '도착점 완료 화면 만들기'
- ④ 캡처 → 노션 제출

결과물

도착점 완료 화면
(실습 1~6 요약 자동 포함)

▶ 명령 실행 사이클

▶ 워크시트 열기

예측 먼저, 화면은 검산 — 채점하면 예측 칸이 잠깁니다

확 인

손 전체 응답 — 번호를 손가락으로

“ 도착점 — ① 설명을 마쳤고 짝의 설명도 들었다 ② 하나만 했다 ③ 모르겠다 — 번호로
”

② 가 많으면 → ② 는 시간 내 교대 완료 유도

오늘 하루 회수

게이트 → 가산기 → 뿔셈 → 플래그 → 제어 → 명령 → 실행

아침의 1비트가 저녁의 프로그램이 됐습니다 — 건너뛴 구간은 없었습니다

마무 리

3일간 수고하셨습니다

부품(그릇) 안의 숫자(내용물)가, 오늘 여러분의 명령으로 움직였습니다.

[강사 확정] 사후 설문

도구(멘티미터 재사용 여부) · 문항 — Day 1 사전 설문과 짝